Test-model abstractions for robotic systems

Hugo Araujo
King's College London
United Kingdom
hugo.araujo@kcl.ac.uk

Ana Cavalcanti
University of York
United Kingdom
ana.cavalcanti@york.ac.uk

Robert M. Hierons
University of Sheffield
United Kingdom
r.hierons@sheffield.ac.uk

Alvaro Miyazawa
University of York
United Kingdom
alvaro.miyazawa@york.ac.uk

Mohammad Reza Mousavi
King's College London
United Kingdom
mohammad.mousavi@kcl.ac.uk

***Abstract*—Robotic and Autonomous Systems (RAS) are multidisciplinary, safety-critical systems whose complex and heterogeneous designs pose significant challenges for verification and validation. While the use of test and simulation models is well established in the robotics community, selecting the appropriate level of abstraction for these models remains a crucial and under-studied challenge. This paper presents an empirical study of how test-model abstraction levels affect RAS verification. Because RAS models are, by necessity, approximations of both the physical platform and the surrounding environment, we first conducted semi-structured interviews with experienced roboticists to identify domain-specific abstraction patterns and assess their usefulness in verification. The insights gained from those interviews were then verified through a controlled experiment designed to obtain quantitative empirical evidence. Specifically, we evaluate the effect of varying abstraction levels on the effectiveness of a search-based testing process, measuring three complementary metrics: fault-detection capability, precision (i.e., absence of false positives), and sensitivity to subtle faults. Based on the combined qualitative and empirical evidence, we propose concrete guidelines for abstracting and progressively refining RAS test-models, including practical guidance for deciding when further refinement yields diminishing returns. Our results demonstrate a strong positive correlation between model refinement and both fault detection and sensitivity, and a strong negative correlation between refinement and false positives. Based on these findings, we distil a set of practical guidelines providing a systematic, step-wise process for constructing and refining RAS test-models to balance verification effectiveness against modelling cost.**

I. Introduction

Developing trustworthy robotic and autonomous systems (RAS) requires rigorous, structured validation and verification techniques. Yet many state-of-the-art testing approaches for RAS remain manual, costly, and offer little or no formal guarantee of their efficiency [1].

As an answer to this challenge, *Model-Based Testing* (MBT) is a family of rigorous testing techniques that exploit formal or semi-formal models to guide the testing process and verify that a system conforms to its design specification. Particularly, test-case generation can be directed to explore and cover the operational space of a test-model systematically, yielding effective test suites with, for instance, well-defined notions of assurance and requirement coverage [2]. Nevertheless, constructing test-models for complex RAS is demanding: despite a long tradition of model development, there is little actionable guidance for producing effective test-models in general [3] and, to the best of our knowledge, none has been proposed specifically for RAS [4].

A central design decision when employing models for testing is the choice of *abstraction level*, i.e., the degree of detail retained in the model. In this work we investigate how to determine the appropriate abstraction for RAS test-models in two complementary senses: (i) the model must be *sufficiently detailed* to generate test cases that are feasible and relevant given the physical and environmental constraints of the system; and (ii), the model must be *abstract enough* that its own construction and verification do not become a prohibitive obstacle. Models that are too coarse can only specify partial behaviours and therefore cannot support comprehensive testing. Conversely, models that exactly replicate the system-under-test (SUT), e.g., models used for full code generation, effectively use the system to verify itself, which may produce false negatives: tests that pass when they should fail. The ideal abstraction level lies between a pure specification model and a full implementation.

Development cost is a further consideration, especially for RAS. Since constructing an exact replica of a physical platform in software is prohibitively challenging, robot (surrogate) models are almost always approximations of both the platform and the environment. It is well known that a model's abstraction level influences its effectiveness, validity, and development cost [5]; in this paper, we go beyond this general knowledge by providing domain-specific patterns of abstraction for RAS and examining the specific effects of these abstractions on the effectiveness of the resulting verification.

Our work provides what is, to our knowledge, the first *empirical* study of test-model abstraction specifically for RAS. We have conducted semi-structured interviews with roboticists to identify how they prioritise aspects of systems or how the test process evolves over time. From the result of the interviews, we identify and analyse the most impactful types of abstraction when modelling robots to help us identify an appropriate level of abstraction when using such models for testing. We further hypothesise that refining test-models result

in more effective models with diminishing returns with respect to their effectiveness compared to more abstract models. To verify our hypotheses, we conducted a controlled experiment in which we constructed a sequence of test-models at increasing levels of refinement for three RAS case studies, generated test suites using search-based algorithms [6], [7], and measured their effectiveness across three metrics: *fault-detection capability* (the proportion of faults detected by the generated test suite); *precision* (the inverse of the false-positive rate, i.e., the proportion of fail verdicts that should actually be pass); and *sensitivity* (the ability to detect faults that produce only subtle deviations from the expected behaviour, quantified via the spatio-temporal distance between observed and expected outputs). Hence, the main contributions of this paper are as follows.

1) We conduct a series of semi-structured interviews with roboticists on the topic of modelling RAS. We enquire about the usefulness of different types of abstraction and what they consider to be the optimal level.
2) We perform a controlled experiment in which we consider the insights obtained in the interviews to develop a series of test-models with varying degrees of abstraction. The test-models are fed to search heuristics for the purpose of testing. In this experiment, we aim to validate our conclusions about the interviews and assess our hypotheses about test-model abstraction levels. We comparing the number of detected faults, the number of false positives, and the sensitivity of the detected faults.
3) We analyse the results we have obtained to identify strengths and gaps in the domain and present recommendations to researchers and practitioners towards developing optimal test-models for RAS. As part of our guidelines, we present a systematic process that progressively refines a test-model across different abstraction types to identify the optimal level of detail for effective testing. The lab package comprising interviews, models, and experiment results are publicly available[1].

We acknowledge three main limitations. First, our experiments refine each test model cumulatively using a fixed order based on component dependencies and interview-derived impact rankings. While this captures the effect of progressively reducing abstraction, it does not isolate the contribution of individual abstraction types or assess alternative refinement orders. Second, our qualitative findings are based on interviews with eight experienced roboticists from the authors' professional network, representing a sample that may not reflect the wider community. Third, our quantitative evaluation includes three stationary or ground-based case studies of moderate complexity. Although selected to span different robot types and abstraction dimensions, they do not represent the full diversity of robotic systems. We therefore present our findings as domain-specific empirical evidence rather than universally generalisable conclusions, and revisit these limitations in Sections IV and V.

[1]https://zenodo.org/records/21181041

A. Research questions

This work aims to answer the following questions:

- **RQ1)** What are the types of abstraction that are perceived to be effective for RAS test models?
- **RQ2)** How do abstraction and refinement influence the fault detection capabilities of generated test suites?
- **RQ3)** How do abstraction and refinement influence the precision of the generated test suites?
- **RQ4)** How do abstraction and refinement influence the sensitivity, that is, the ability to detect more subtle faults of the generated test suites?

We make use of the insights obtained during the interviews to answer RQ1. As for RQ2, RQ3, and RQ4, we conduct a controlled experiment and measure the number of true positives (detected faults), false positives (incorrectly reported faults) and the sensitivity to the difficulty of faults (the likelihood of detecting "more obvious" faults at the same rate that it can detect "less obvious" ones). We expect subsequent iterations of the test-model will lead to detecting more faults, with fewer false positives, and identifying difficult faults more easily.

B. Structure of the paper

This paper is organised as follows. In Section II, we discuss related work. In Section III, we describe the notations and techniques used in our work, and a running example. In Section V, we describe the design and the execution of our controlled experiments. In Section VI, we present some guidelines to roboticists with respect to abstraction levels. Finally, in Section VII, we draw some conclusions and point out the directions of our future research.

II. Related work

Model-based testing has been traditionally developed and used in the context of embedded systems [8]. There are recent advancements in the field in order to use models of robotic and autonomous systems for test-case generation, e.g., for the control software [9] and for system dynamics [10], [11], [12], [13]. None of these recent advancements discuss the quality of test-models or processes to come up with them.

On the topic of abstractions, a general introduction into MBT that explains their role is presented by Utting et al. [3]. Moreover, Prenninger and Pretschner [5] conduct a study on the types of abstraction that can be applied to MBT in general. They provide a list of useful abstractions and some guidance into building test-models. Our work follows a similar approach but with focus on RAS, thus identifying additional types of abstractions. Further, we provide qualitative and quantitative evidence on the usefulness of each type.

Afzal et al. [14] conduct a study with robotics developers to understand the key challenges they face when using simulation for testing and test automation. Their results indicate that simulation is a popular tool amongst developers; however, the reality gap (the inherent gap between digital and physical robots) remains a major hurdle without satisfactory solutions. This work is an attempt to tackle this challenge by providing insights and guidelines.

Recent work has broadened the scope of simulation-based testing for autonomous systems, providing strategic roadmaps that highlight test automation and quality assurance as key open challenges [15]. In particular, the reality gap is identified as one of the most pressing unsolved problems in this space, with domain randomisation and simulation-to-reality strategies proposed as partial mitigations [15]. Complementing this view, Ortega et al. [16] demonstrate that managing the huge variability of test scenarios is itself a major barrier for robot software engineers, and propose domain-specific languages to specify and automatically generate diverse virtual environments for mobile robots. They have shown that only complex scenarios were able to surface a bug present in a widely used navigation stack, which remained undetected for over a year. This shows that the level of environmental detail in a test model can have a direct impact on which faults can be exposed. None of these contributions, however, provide a systematic process for determining how much detail is enough, which is the specific gap the present paper addresses.

Our study also relates to two adjacent lines of work. First, because an exact software replica of a physical platform is infeasible, RAS test-models are necessarily *surrogate* models: approximations of the platform and its environment. The mismatch between such surrogates and reality is precisely the reality gap discussed above [14], [15], and the abstraction levels we study can be seen as a controlled way of navigating this gap for the purpose of testing. Second, in the broader cyber-physical systems (CPS) literature, model abstraction is a well-recognised trade-off between fidelity and analysability [5], [10], [13]: richer models capture more behaviour but are costlier to build and harder to reason about. Our contribution is complementary and more specific: rather than treating this trade-off in the abstract, we provide domain-specific abstraction patterns for RAS and empirical evidence of how each abstraction dimension affects fault detection, precision, and sensitivity, thereby grounding the fidelity/cost trade-off in measured verification outcomes.

III. Background

This section describes our approach to modelling of robotic designs (Section III-A) and a brief overview of MBT approaches (Section III-B).

A. Design models of robots

When modelling robotic systems, there are three main parts of the system that are tightly integrated but function separately: the controller (i.e., the software), the platform (i.e., the hardware), and the environment.

- **Controller.** Controller models allow the specification of cyclic-based control software, which abstracts the services provided by the robotic platform in terms of variables, events, and operations. It can change the state of the real world through the generation of control signals that are sent to the platform via interfaces.
- **Platform.** The platform model includes the physical parts of a robot, including sensors and actuators. It define interactions with the environment and the software controller and can be seen as the interface between both. The specific combination of links (rigid 3D parts) and joints, which connect links, dictate the trajectories and motions that are physically allowed by the robot.
- **Environment.** The environment represents the real world that interacts with the robot. Capturing realistic physical characteristics of the environment in a model is a non-trivial task, and can be done via modelling scenarios by specifying entities (alongside their properties) such as the terrain, obstacles, and other robots.

In this work, we employ the RoboStar framework [17] for the design model and development of robotics systems, which is based on a family of modelling languages, including RoboChart [18] and RoboSim [19]. Additionally, we use CoppeliaSim [20] to simulate the systems. CoppeliaSim (formerly V-REP) is a widely used robotics simulator which supports a variety of robot description formats, including SDF, and programming languages, including C++. While these tools were selected for their maturity and usage, we acknowledge that the use of specific modelling languages and simulation environments may influence the generalisability of our findings. In particular, the abstraction patterns and refinement guidelines derived in this work are grounded in the semantics of RoboChart and RoboSim; although we expect the underlying principles to transfer to other modelling frameworks, their applicability to substantially different notations or simulators remains a subject for future investigation.

B. The MBT approach

Model-based testing (MBT) is an approach to facilitate the automation of the testing process. Test cases are generated from a specification model (i.e., a test-model) and are then executed on the SUT and the verdict (pass or fail) is given, thus verifying compliance with system requirements.

Generating test inputs for continuous systems can usually be seen as an optimisation problem, which can be solved using search techniques [21]. A search problem is one in which optimal or near optimal solutions are sought in a search space of candidate solutions, guided by a fitness function that distinguishes between better and worse solutions [22]. Many search-based algorithms, with specific advantages and disadvantages, can be found in the literature. Examples include Simulated Annealing [6], Genetic Algorithm [7], Ant Colony Optimisation [23], and Particle Swarm Optimisation [24].

IV. Semi-structured Interview

We conducted a series of semi-structured interviews on the topic of testing using robot models. The interviewees were provided with background information about the use of models in testing along with the set of six questions that were posed to all participants. In the context of the interview, we have considered a model as a description of a robotic system or its components. This description can be formal, such as state machines and behaviour trees, or informal, such as informal description of Roboto Operating System (ROS) [25] nodes.

A. Methodology

The objective of this semi-structured interview is to explore and understand the methodologies, criteria, and challenges related to the evolution of testing processes currently used in robotic systems. Specifically, we investigate which factors influence how roboticists prioritise different aspects of the system during testing, how their testing strategies evolve throughout the system development lifecycle, and how these prioritisation choices affect the types of faults detected in the final system. To this end, we ask roboticists questions on the topic of robotic testing to draw conclusions related to the evolution of test-models.

The participant selection process comprises roboticists or computer scientists with at least two years of experience in developing and testing robots. We've sampled the participants by identifying, amongst our professional network, interviewees who are sufficiently knowledgeable and experienced to answer our questions. We have catered for diversity and experience in our selection. As a result, we have identified 10 participants (3 female and 7 male) to ensure a range of perspectives while maintaining manageability. All of the participants have a doctorate in either computer science or robotics engineering with at least five years of experience working with robots. Four of them also have at least three years of experience as an industry practitioner.

The format of the study follows a semi-structured interviews, which allows for in-depth conversations with flexibility to thoroughly explore topics in which the participant has more expertise. Each interview lasted between 30 and 60 minutes, and was conducted via a video line. The participants were informed about what to expect in terms of the purpose of the study, background information, the questionnaire itself, and expected length of time and that a non-identifying, verbatim, transcript of the interview, was to be made available online as an artefact of this study. The questions asked to the participants were as follows:

- What aspects of the system behaviour do you prioritise in testing? Can you provide the rationale behind the prioritisation choices?
- Is there any planning for prioritising certain aspects of the system? Does this plan change from system to system? If so, how? How can you justify the plan?
- Does the type of test change as the system matures? How do you gradually add more details to the testing process?
- Can you imagine any changes to the types of details (or abstractions) that are prioritised? If you don't, what is the reason behind it? If you were to classify the types of changes, would they match with the types of abstractions highlighted above?
- What types of faults are observed in the final system? Could these faults have been prevented in an earlier testing phase by prioritising certain aspects of the system?

Given the semi-structured nature of the interview, each participant was also asked additional questions depending on the flow of the conversation, either to clarify or to explore a concept. A transcription of the interviews is available in the lab package.

We analysed the interview data using a thematic coding approach. Each verbatim transcript was independently read and coded, with codes assigned to segments referring to a particular aspect of a robotic system or its modelling (e.g., timing, sensing, communication). Related codes were then grouped into higher-level themes, which we consolidated into the abstraction types reported in Section IV-C; the biological abstraction type emerged directly from this process as a category we had not anticipated. Finally, we ranked the abstraction types from most to least impactful according to the degree of agreement among interviewees regarding their importance for verification, and we distilled, for each type, the recurring guidance on when to abstract and when to refine. The complete coded transcripts are available in the lab package.

B. General Insights

The general consensus amongst the interviewees is that investing time in robust system design can significantly reduce the modelling burden during verification. For instance, several interviewees estimated that a well-designed communication layer can eliminate the need to model message loss entirely, reducing the complexity of the corresponding test-model. More broadly, the interviewees agreed that abstractions should be applied wherever possible: in their experience, abstracting non-critical components (e.g., logging subsystems, secondary sensors) can reduce model development time by a third, without measurably affecting fault detection rates. The main exceptions to this principle are functionality and safety-critical components, which all interviewees agreed must always be modelled faithfully. Abstracting these was reported to account for the majority of missed faults in prior projects by one interviewee.

When it comes to the environment, accurate modelling is something that is hardly ever done. One way to handle the environment is to build specific scenarios for testing. Another way is to build simulations where only the most essential aspects of the environment are considered, such as lighting condition for image processing or the presence of obstacles. In some cases, it is also possible to apply nondimensionalisation which helps to simplify the equations in the environment. The lack of the testing involving environmental conditions can sometimes be justified by hardware certification or by assuming likely operation conditions as a premise to validity.

Lastly, the interviewees have not mentioned any noticeable pattern or systematic approach to determine the optimal level of abstraction. That is, there are no general insights for when an aspect of the robot has enough detail and should not be refined further. The interviewees consistently indicated that determining the appropriate level of abstraction relies heavily on the judgement of domain experts, with no systematic methodology in place; a process that is, by their own admission, largely ad hoc.

C. Insights on abstraction types

Below, we summarise the discussion for each type of abstraction identified in this work, as well as the additional types raised by the interviewees. We have sorted the abstractions below from most to least impactful according to the answers we have received.

1) Functionality Abstraction: The roboticists we interviewed generally agreed that modelling functionality is the most important aspect of a model and the most domain-agnostic type of abstraction. The reason is that functionality abstraction leads to simplifying or removing important traits of a system, potentially overlooking crucial intricacies (such as interaction between features) that are vital for comprehensive testing. Unfortunately, not much insight has been gained on this type of abstraction, as the interviewees agreed that it is important to model functionalities faithfully. To provide actionable guidance, we summarise the interview findings for each abstraction type in terms of when it is appropriate to abstract and when further refinement is warranted, as these represent the two key decisions a practitioner must make when constructing a test-model.

- *When to abstract:* It can be convenient to not model features when they do not influence the robot's control (e.g., camera feed that is only used for logging). This, however, can lead to untested behaviour, such as the interaction between the controller and the camera.
- *When to refine:* Features that are linked to the primary objectives should receive the most attention. General consensus is that when modelling a feature, it is important to model the exceptional cases as well as the common behaviour as they are likely to reveal faults.

2) Physical abstraction: Faithfully modelling the physical aspects of a robot is considered an important aspect of testing, albeit less so than functionality. It provides a comprehensive understanding of the motion of the robot and how it will behave in its environments. In simulation environments, this enables engineers to anticipate and address potential challenges or limitations. Roboticists tend to split physical motion between kinematics and dynamics: kinematic models describe the robot's motion (e.g., velocity) without considering the forces involved, while dynamic models take into account the forces (e.g., friction) and torques acting on the robot.

- *When to abstract:* Generally speaking, modelling physical motion facilitates the validation of control and navigation algorithms. However, the level of detail needed in a test-model largely depends on the importance of accuracy of the robot's task. In case where precision is not the primary concern, then the dynamics (which is incredibly challenging to model precisely) acting on the robot and its environment can be abstracted away or simplified into linear models. As an example, a drone whose goal is to scope out an area does not need a faithful dynamic model of air resistance and wind conditions. Furthermore, certain aspects of the dynamics of a robot can be better tested in performance analysis such as finite element analysis [26].
- *When to refine:* Roboticists tend to optimise the physical abstraction by only modelling the necessary kinematics for the operation of the robot, since these are relatively straightforward to model accurately. There are a few special cases where a higher level of detail is warranted. Firstly, if the robot operates in high-risk environments (e.g., ground robot that needs to traverse uneven terrain) and, secondly, if the robot physically interacts with a human; in both cases, a detailed model of the motion and forces is advisable. As for the physical aspects of components that are not related to motion, a more faithful modelling is recommended if such components directly affect mission planning or high-level decision of the robot's behaviour. For example, an accurate modelling of the battery is necessary if it has an impact on the mission control (e.g., the robot must return to a charging station whenever the battery is low). Lastly, by incorporating a more realistic motion modelling into the testing process, developers can identify inefficiencies when trying to optimise tasks and components when comparing different configuration models.

3) Temporal abstraction: Temporal abstraction can be referred to in two ways: (i) external, which indicates the ability of a robot to execute physical tasks with correct timing and (ii) internal, which refers to software related performance and concurrency (e.g., racing conditions). Unlike functionality abstraction, the usefulness of temporal abstraction is heavily dependent on the domain of the robot and on the sensitivity of features.

- *When to abstract:* When it comes to external aspects of temporal abstraction, modelling the precise timings of a robot is a challenging task due to the naturally occurring issues with communication and mechanical delays. Temporal deviations are quite common and accepted when dealing with robots and should be taken into account the verdict of the verification process rather than on the model. As for the internal aspects, a common type of abstraction is to cater for the ordering of events/messages in the models. Due to asynchrony and racing conditions often present in robot design, it is possible for messages to arrive out of order. The test-models should take this into account whenever features rely on working with the most up-to-date messages (e.g., the frames received from a camera output can be be received out of order and thus, it is important to know which is the latest frame).
- *When to refine:* In certain cases, timing issues are vital and a pivotal source of issues for specific features. Such features are often linked to safety, for example, IMU stabilisation and air bag deployment features. Hence, the usefulness of temporal abstraction is heavily reliant on how time-sensitive specific features are and, therefore, the advice is to not use this type of abstraction in the parts of the model that are time-critical. Another case in which

the interviewees considered more faithful modelling of temporal aspects to be warranted is in high-performance systems, such as autonomous cars.

4) Data Abstraction: We refer to data abstraction as the simplification of sensor output that is handled by the software interface.

- *When to abstract:* Complex sensor data are generally not modelled in detail. For instance, image feed is typically processed by separate (e.g., third-party) algorithms for feature extraction or object detection. These algorithms have their own testing methodology. Typically test-models do not make use of raw feed and only take into account whether something has been identified or not. Another possible technique for data abstraction is sensor fusion [27]: instead of modelling multiple sensors separately, one can choose to model a fusion of the sensors data, which can simplify the modelling process.
- *When to refine:* Sensors that output simple data such as distance sensors or GPS coordinates are typically fully modelled. As for more complex sensors used in object detection, such as lidars and cameras, there are a few practices that can turn a simple model into a more realistic one. For instance, one can incorporate a probability distribution into the detection mechanism to mimic noise and occlusion. Some robots incorporate probabilistic reasoning to determine the most appropriate action in the absence of sufficient sensor data. This probabilistic reasoning should be incorporated into the test-model as it affects the decision making of the robot.

5) Communication Abstraction: Communication abstraction can be divided into internal and external communication. Internal communication refers to the exchange of messages (either synchronously or asynchronously) between internal components, whilst external communication deals with contact between the robot and external agents.

- *When to abstract:* According to the interviewees, internal communication is the most used form of abstraction and it has the least impact. Most of the functionalities in robots are asynchronous by design and most systems are robust enough to cater for message loss by sending constant communication updates. Hence, even abstract models with perfect communication are generally adequate. An example of that is the image feed from a camera to the image processing unit where single frames being dropped are not important.
- *When to refine:* It is generally important to model external communication accurately, especially in the case of wireless communication. The reasoning is that external communication is much more prone to delays, and it typically has a major impact on the behaviour of a robot, such as when it needs to receive authorisation or teleoperation commands. Hence, one should take message loss into account and model the communication failure scenarios. The only exception is if the external communication is only used for logging.

6) Human Abstraction: Modelling humans is an incredibly complex task. Abstractions in the human models is very common whenever analysing/verifying robotic systems with a human-in-the-loop. Human-robot interaction can be predicted by computing expected actions in behavioural models. Evidently, no behavioural model can fully capture the nuances of the human behaviour; hence, they only capture the most likely scenarios. If physical interaction between the human and the robot is expected, then the physics of human motion is also modelled to prevent use of excessive force.

7) Biological Abstraction: Biological abstraction was mentioned by a participant as an additional type of abstraction previously not considered by us. This can be useful in two contexts; whenever one needs to replicate biological mechanisms (e.g., a patient of a robotic surgery) and, also, when one needs to simplify neural models for testing and simulation (e.g., using offline learning instead of real-time learning).

We consider human and biological abstractions to be outside of the scope of this study.

V. Controlled Experiment

In this section, we discuss our controlled experiment regarding three case studies. We develop a set of test-models at different levels of abstraction for each system, and execute three different trials designed to assess our research questions. For each system, we present the results followed by a statistical analysis. Lastly, we discuss our conclusions and the threats to their validity.

A. Case Studies

In this section, we describe our three case studies and how we gradually refined each one to reduce abstractions. We mainly focus on removing functionality and physical abstractions, but other types of abstractions are also considered, such as communication abstraction, depending on the case study.

The three systems span recognised axes of variation (e.g., autonomy) while remaining tractable for a controlled experiment. Specifically, they differ along (i) the autonomy dimension defined in ISO 8373:2021 [28], comprising a remote-controlled (teleoperated) robot (the RC car), an automatically-controlled manufacturing arm, and an autonomous mobile robot (the solar panel cleaner); (ii) their dominant core functionality, namely object manipulation, environment navigation, and human-in-the-loop/communication, respectively; and (iii) the abstraction dimensions they exercise which, taken together, cover all the types studied in this work at differing intensities (Tables I–III). This spread is intended to maximise coverage of the phenomena under study rather than to sample any single category exhaustively.

The refinement of each test-model is cumulative: each model retains all features of its predecessor and adds detail. This ordering is by design and is governed by two considerations. First, component dependencies: many refinements presuppose component/features introduced by earlier steps and therefore cannot precede them. For example, gripper angular velocity (Table I, step 6) requires the gripper rotation

introduced in step 5; image noise (step 8) requires the detection sensor introduced in step 7; and communication delay (Table III, step 8) requires the communication channel introduced in step 7. Second, the ordering prioritises functionality first, consistent with the impact ranking elicited in our interviews (Section IV), and mirrors how practitioners incrementally construct test-models in practice. We discuss the limitation this cumulative design imposes on isolating individual abstraction effects in Section V-C.

1) Manufacturing arm: In this case study, we consider a robotic arm whose goal is to place two identically-sized manufacturing pieces precisely on top of each other. The system works as follows. Pairs of pieces are put on a conveyor belt, which moves at constant speed. A camera detects the pieces (along with their location and rotation) whenever a pair approaches the arm; the arm then picks up one of them, rotates it, and places it on top of the other. It then resets its position to a neutral one. The arm itself has four components: three links (base, forearm, arm) and an end-effector (a gripper). Joints connect each of the links.

Table I shows the refinement steps of the robotic arm model. The initial and most abstract one comprises only the end-effector and the pieces to be assembled. The behaviour is that the piece is grabbed and instantly moved to be on top of the other piece. This forms the basis of our manufacturing robot system. Effectively, we disregard most of the physical movement constraints present in a real robotic arm. This suffices to describe the starting state and the end goal of the system, which enables a simple input/output behaviour.

TABLE I: Refinement steps of the robotic arm model

Model	Type of Abstraction	Added Behaviour
1	Functionality	Grabbing and moving pieces
2	Functionality	Behaviour of the arm
3	Physical	Forward and Inverse kinematics
4	Temporal	Movement speed of arm and pieces
5	Functionality	Instant rotation of the gripper
6	Physical	Gripper angular velocity
7	Functionality	Detection of pieces
8	Data	Gaussian noise image
9	Physical	Weight dynamics

Subsequent models progressively refine this baseline, as summarised in Table I: they add the correct end-effector behaviour, forward and inverse kinematics (matching a delta robotic arm), temporal and angular-velocity constraints on the motion, detection of the pieces with added image noise, and finally weight dynamics.

2) Solar Panel Cleaner: The solar panel cleaner is an autonomous vacuuming robot whose goal is to traverse solar panels for cleaning purposes [29]. The environment is an array of solar panels (with a degree of inclination compared to the ground) that are connected via a bridge section. The robot must cover the surface of each solar panel completely before moving to the next one; further, it must automatically detect the edges of the panels and the bridge sections to avoid falling off the panels. Lastly, it must return to base when battery levels are low to recharge and, also, the panels must not be placed with an inclination that affects the robots task.

For this system, we prepared seven test-models (Table II). The most abstract one models the environment (the solar panels) and represents the robot as a dot that moves regardless of battery condition and inclination; subsequent models add velocity kinematics, battery-driven recharging, rotation behaviour, friction (heavily impacted by the panel inclination), and finally a distance sensor for edge detection to avoid falling off the panels.

TABLE II: Refinement steps of the Solar Panel Cleaner model

Model	Type of Abstraction	Added Behaviour
1	Functionality	Robot movement
2	Physical	Velocity kinematics
3	Functionality	Battery functionality
4	Functionality	Robot instant rotation
5	Physical	rotation angular velocity
6	Physical	Weights and Dynamics (Friction)
7	Data	Distance sensor (avoid falling)

3) RC Car: For our third case study, we model an RC Car, which is a remotely controlled miniature vehicle. The model comprises torque and traction modules to control the vehicle's dynamics as well as a Controller Area Network (CAN) node used to transmit sensor data and receive commands from a controller for steering, accelerating and breaking. The vehicle also has a battery that provides power to the electric motor. The goal of the system is to adequately respond to the inputs given from the controller device. We provide eight different test-models for this system, shown in Table III.

TABLE III: Refinement steps of the RC Car model

Model	Type of Abstraction	Added Behaviour
1	Functionality	Vehicle Acceleration/Deceleration
2	Physical	Torque vectoring sensitivity
3	Functionality	Wheel steering control
4	Physical	Traction Control Sensitivity
5	Functionality	Power loss and Battery Drain
6	Physical	More realistic battery decay
7	Communication	Remote control (CAN send and receive)
8	Temporal	Communication delay

The most abstract model comprises the base of the car receiving inputs from the controller directly (as if wired). Subsequent models (Table III) add the motor, wheel steering and traction control, a power unit with an increasingly realistic battery decay, and finally remote communication behaviour via CAN nodes and communication delay.

B. Experiment Design

In this section, we explain the experiment design. Particularly, we describe the methodology (Section V-B1), our metrics (Section V-B2), and hypotheses (Section V-B3).

1) Methodology: The testing process of this experiment follows a standard MBT approach (see Section III-B). A test is defined by the combination of inputs, outputs, and the test oracle. The inputs are generated using a search-based approach - using Simulated Annealing (SA) and Genetic Algorithms (GA) as described in [30] - and then are executed on the

system-under-test (SUT), which yields the outputs. We have generated 50 inputs (i.e., test cases) to evaluate this system. Both the inputs and the outputs are represented by a trajectory, which, essentially, captures the valuation of system variables over time. Thus, input and output trajectories capture the history of input and output variables throughout the execution of the system, respectively. The goal of the test is to evaluate whether the resulting output trajectory, given a particular input trajectory, is correct.

To obtain the verdict of a test, we make use of a test oracle, which checks whether the output from the SUT conforms to the expected output (given, for example, by the test-model). For that, a *conformance notion* is (a mathematical formulae that specifies compliance between the SUT and a design model) can be used. The conformance notion employed in this work [31] is parametrised by a pair (τ, ϵ) that gives an allowed tolerance. In testing RAS, due to the inherent imprecision in physical sensing and actuation devices used in the testing apparatus and also in the SUT, the observed behaviour often deviates slightly, up to a specified margin of error, in time and value with respect to the model [32]. Hence, we have τ, which is a bound on how much the timing can differ from that specified and ϵ, which is a bound on how far the system output (or position) can deviate from that given by the requirements function.

We define specific values of maximum allowed deviation (i.e., τ and ϵ) for each system. The values we have chosen are based on pilot experiments and domain knowledge. Any deviation above τ and ϵ is considered a failure and, thus, the test fails. In the case of the robotic arm, we consider a behaviour to be faulty when there is a discrepancy of 1 or more centimetres with respect to the final position of the manufacturing pieces. In other words, if the centre of a piece ends up less than 1 cm away from the centre of the matching piece, the test is given the **pass** verdict. As for the two other case studies, a distance of 10 cm for the trajectory of the Solar Panel Cleaner, and 10 degrees of steering angle or 1 m/s speed difference for the RC Car were considered. For all examples, the maximum allowed temporal deviation (i.e., τ) is 1 second. Two domain experts were involved in the decision of such values. One with expertise in the development of robotics and the other with expertise in verification of RAS.

Although the test oracle determines whether a simulation run corresponds to a fault, there remains the question of how we choose *which* simulations to run. In this work, we use two multi-objective search-based algorithms, namely, Simulated Annealing and Genetic Algorithm, for test case generation. The algorithms employ 'fitness functions' to optimise the search. First, for the expected output o_E we have a function f_o that estimates how 'close' the output of a simulation run o_S was to failing. If o_S can be different from o_E (and, thus, result in non-conformance) then an input that maximises f_o should achieve this. The second fitness function is a coverage measure that represents the proportion of the states in the test-model that are covered by the test suite (set of input trajectories). And for the third one, we use a measure of how *diverse* the test suite becomes when we add a new input, basing diversity on the Euclidean distance between the new input and those previously added. The aim is to maximise diversity since there is evidence that diverse test suites tend to be effective [33].

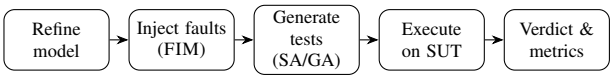

Fig. 1: End-to-end experimental workflow. Test-models are refined manually (Tables I–III); faulty variants are produced automatically with FIM; and search-based test suites (SA/GA) are then generated, executed on the system-under-test, and evaluated against the metrics.

To evaluate the hypotheses, we perform controlled experiments on our implementation, which has been thoroughly simulated, tested, and used and is believed to be of high quality. To asses two of our research questions, namely RQ2 (fault detection capability) and RQ4 (sensitivity), we produce faulty versions of this believed-to-be-correct implementation. For that, we use our own domain-specific mutation operators [34].

We distinguish two separate stages of our methodology. First, the refinement steps (i.e., the sequence of test-models at increasing levels of detail defined in Tables I–III) were authored manually by the authors. Second, for the mutation analysis used to answer RQ2 and RQ4, the faults were injected automatically into the believed-to-be-correct implementation using FIM, a mutation tool for Simulink [35]. We applied 10 instances of each mutant operator (described below) per model. The end-to-end workflow is summarised in Figure 1.

The *Delay Operator* simulates the introduction of delays into the model, while the *Noise Operator* adds noise to signals within the Simulink model. The *Packet Drop* operator replaces the value of a variable with an alternative one. Finally, the *Logical Operator Replacement* and *Arithmetic Operator Replacement* mutators each substitute an operator with another of the same type. These operators were selected on the basis of their compatibility with our models.

In total, 50 faults were inserted per model. No higher-order mutants were used, i.e., at most one fault was introduced at a time [36], and all mutants are confirmed to be non-equivalent by inspection of each operator's effect on the model output. The *same* mutant set is reused across all refinement levels of a given case study, ensuring that mutation scores are directly comparable across models. The fault types used in this work cover four categories: *kinematic* faults (related to motion), *dynamic* faults (related to forces), *statement* faults (involving mathematical operators and variable valuations), and *temporal* faults (involving time shifts). These categories are designed to represent classes of failures observed in practical circumstances, including sensor noise, coding errors, and system slowdowns.

Table IV relates each mutation operator to these four fault categories. Some operators map to a single category by construction: the Delay operator produces temporal faults

(time shifts), while the logical and arithmetic operator replacements produce statement faults (altered operators or variable valuations). Others are context-dependent, since the category they yield depends on the component to which they are applied: an arithmetic replacement or a signal-noise perturbation within a kinematic computation manifests as a kinematic fault, whereas the same operator within a force/torque (dynamic) computation manifests as a dynamic fault. The Packet Drop operator substitutes a variable's value (a statement fault) but can manifest as a temporal fault where it disrupts message timing.

TABLE IV: Mapping between mutation operators and fault categories. ✓ marks an operator's primary fault class; ○ marks a class it can also produce, depending on the model component to which it is applied.

Operator	Kinematic	Dynamic	Statement	Temporal
Delay				✓
Noise	○	○	✓	
Packet Drop			✓	○
Logical Op. Replacement	○	○	✓	
Arithmetic Op. Replacement	○	○	✓	

To answer RQ2, we follow a direct approach. First, we generate the test suites from our test-models and feed these to the faulty implementations. Afterwards, we collect the outputs, analyse them, and compute the mutation score. As for RQ3, the experiment methodology does not employ mutation testing. Instead, we generate test cases and compare the expected output from the test-model against that of the correct implementation, which, in this case, we assume to be the ground truth. If non-conformance is detected then, in this case, we classify it as a false positive.

For RQ4, we estimate the impact of each mutant and classify them from most subtle to least subtle. For instance, a mutant that modifies the value of a constant from 9.8 to 9.9 is likely to result in less obvious failures than changing the same constant to -1. In practice, however, this will depend on the context and on the input itself. In order to estimate the impact, each mutated system is exercised using random inputs and the average distance between the correct and the mutated system outputs is computed. The closer the deviating behaviour is from the expected one, the more difficult it is for a test suite to detect the fault and, thus, the sensitivity to faults is measured by checking whether more subtle faults can be detected. Here, we make use of the same mutants used to answer RQ2 and we follow a similar process for test-case generation and execution. However, this time, our analysis checks which test-models managed to detect the most subtle mutants.

2) Metrics: As mentioned, in this experiment, we make use of the following metrics: (i) the number of faults detected, (ii) the number of false positives (i.e., incorrect **fail** verdicts), and (iii) the distance to the correct output (i.e., sensitivity).

For each case study, we have one correct SUT, a set of mutations (injected faults - see Section V-B1) and several test-models (see Section V-A). The first metric quantifies the fault detection rate by the test suite generated from each test-model. Detecting more faults is generally the goal of any test suite. For a fail verdict, we manually validate whether it is an actual fault or it is a false positive. Lastly, we compute the Euclidean distance between the observed and expected behaviours. The higher the distance, the easier a fault can be identified.

3) Hypotheses: We have defined one hypothesis for each of our research questions and they are presented in Table V.

TABLE V: Experiment hypotheses.

Hypothesis A	
H_{A0}	$MS_{LRTM} \geq MS_{MRTM}$
H_{A1}	$MS_{LRTM} < MS_{MRTM}$
Hypothesis B	
H_{B0}	$FN_{LRTM} \leq FN_{MRTM}$
H_{B1}	$FN_{LRTM} > FN_{MRTM}$
Hypothesis C	
H_{C0}	$SFD_{LRTM} \leq SFD_{MRTM}$
H_{C1}	$SFD_{LRTM} > SFD_{MRTM}$

Hypothesis A (RQ2) concerns fault detection: it compares the mutation score (MS) of less-refined test-models ($LRTM$) against more-refined ones ($MRTM$). The null hypothesis (H_{A0}) states that $LRTM$s achieve a higher or equal MS than $MRTM$s, which the experiment aims to refute; a corresponding alternative (H_{A1}) is also defined.

Analogously, Hypothesis B (RQ3) concerns precision, contrasting the number of false positives (FP) produced by an $LRTM$ against those of an $MRTM$.

Lastly, Hypothesis C (RQ4) concerns sensitivity, comparing the number of subtle faults detected (SFD) by an $LRTM$ against those of an $MRTM$.

C. Results

In this section, we present the results of our experiment.

1) Mutation Score: The first set of results concerns the mutation scores, shown in Table VI.

TABLE VI: Mutation score

Model	Robotic Arm		Solar Panel Cleaner		RC Car	
	S.A.	G.A.	S.A.	G.A.	S.A.	G.A.
1	20/50	21/50	17/50	17/50	14/50	14/50
2	32/50	32/50	19/50	19/50	16/50	16/50
3	34/50	34/50	39/50	36/50	27/50	27/50
4	40/50	40/50	42/50	41/50	31/50	32/50
5	40/50	41/50	43/50	43/50	42/50	43/50
6	42/50	41/50	43/50	45/50	42/50	43/50
7	47/50	46/50	43/50	45/50	45/50	46/50
8	47/50	46/50	N.A.	N.A.	45/50	46/50
9	47/50	47/50	N.A.	N.A.	N.A.	N.A.

The mutation score for each test suite generated from each model shows the number of mutants detected over the total number of mutations inserted. The higher this number, the more effective the test suite in detecting faults. Here, the results indicate that there is an increase in the levels of fault detection capabilities with the reduction of abstraction. Interestingly, whenever a model reduced its level of functionality abstraction, this resulted in the greatest impact on the number of mutants detected. Additionally, we note how the detection levels plateau as the models become more detailed. We write "N.A." if there is no test-model of that level for that case study.

2) False positives: In this part of the experiment, from each test-model, a total of 50 test cases were generated (25 using simulated annealing and 25 using genetic algorithm). We have checked how many of them fail against an implementation that has been extensively tested and has no injected faults.

TABLE VII: False positives

	Robotic Arm	Solar Panel Cleaner	RC Car
Model 1	31	25	42
Model 2	27	16	37
Model 3	14	15	35
Model 4	13	15	30
Model 5	13	4	28
Model 6	6	4	16
Model 7	6	3	12
Model 8	6	N.A.	11
Model 9	5	N.A.	N.A.

As shown in Table VII, we notice a pattern of fewer false positives as we further refine the test-models. This time however, the biggest impact on false positives was driven by removing physical abstractions.

Our intuition is that when we apply our search-based strategy to the more abstractly defined robots, the search assumes they can behave in certain ways that the actual robot cannot. This leads to less reasonable inputs generated when the search is being carried out, as this approach works by finding the optimal (or extreme) cases. For instance, consider the robotic arm. Due to the abstractions in some of the test-models, the location of the pieces are detected with perfect precision. This leads to the correct assembly of pieces even if, in the input, the pieces were randomly stacked on top of each other. However, this stacking of pieces can lead to issues in the actual SUT, as the camera may not detect pieces that are in partial view. The difference in output results in a fail test that is classified as a false positive, as the input had unreasonable expectations.

3) Difficulty in detecting faults: To answer RQ4, we categorise mutants in quartiles based on their average deviation, placing those that result in the 25% least subtle faults in the first quartile. Analogously, the mutants that result in the 25% most subtle faults are put in the 4th quartile.

TABLE VIII: Sensitivity to faults - Robotic Arm

M	Sim. A				Gen. A			
	1st	2nd	3rd	4th	1st	2nd	3rd	4th
1	06/13	05/12	04/13	05/12	07/13	06/12	03/13	05/12
2	08/13	10/12	09/13	05/12	09/13	09/12	09/13	05/12
3	08/13	10/12	09/13	07/12	08/13	10/12	09/13	07/12
4	08/13	11/12	11/13	10/12	08/13	11/12	11/13	10/12
5	08/13	11/12	11/13	10/12	09/13	11/12	11/13	10/12
6	09/13	11/12	12/13	10/12	09/13	11/12	11/13	10/12
7	13/13	12/12	12/13	10/12	13/13	12/12	11/13	10/12
8	13/13	12/12	12/13	10/12	13/13	12/12	11/13	10/12
9	13/13	12/12	12/13	10/12	13/13	12/12	12/13	10/12

The results are shown in Tables VIII, IX, and X. They indicate that, for the RC Car and the Solar Panel Cleaner models, there is a significant gap between the number of subtle faults (1st quartile) detected by the least and most refined test-models. The results also show that removing physical, communication, and data abstraction helps identify the more

TABLE IX: Sensitivity to faults - Solar Panel Cleaner

M	Sim. A				Gen. A			
	1st	2nd	3rd	4th	1st	2nd	3rd	4th
1	07/13	05/12	04/13	01/12	07/13	05/12	04/13	01/12
2	07/13	05/12	04/13	03/12	07/13	05/12	04/13	03/12
3	11/13	10/12	11/13	07/12	11/13	10/12	10/13	05/12
4	13/13	11/12	11/13	07/12	13/13	11/12	11/13	07/12
5	13/13	11/12	11/13	08/12	13/13	11/12	11/13	08/12
6	13/13	11/12	11/13	08/12	13/13	11/12	11/13	10/12
7	13/13	11/12	11/13	08/12	13/13	11/12	11/13	10/12

TABLE X: Sensitivity to faults - RC Car

M	Sim. A				Gen. A			
	1st	2nd	3rd	4th	1st	2nd	3rd	4th
1	04/13	06/12	02/13	02/12	04/13	06/12	02/13	02/12
2	04/13	06/12	03/13	03/12	04/13	06/12	03/13	03/12
3	09/13	10/12	04/13	04/12	09/13	10/12	04/13	04/12
4	09/13	10/12	07/13	05/12	09/13	10/12	08/13	05/12
5	13/13	12/12	10/13	07/12	13/13	12/12	11/13	07/12
6	13/13	12/12	10/13	07/12	13/13	12/12	11/13	07/12
7	13/13	12/12	11/13	09/12	13/13	12/12	12/13	09/12
8	13/13	12/12	11/13	09/12	13/13	12/12	11/13	09/12

subtle mutants (3rd and 4th quartile) that had not been detected through functionality refinement alone. This shows that less refined test-models can be used to detect less subtle faults; however, the more subtle faults can only be detected by more refined test-models. In the case of the first system (the Robotic Arm), the results still match our hypothesis, but the correlation is weaker.

4) Statistical correlation: To provide some statistical analysis, we compute the correlation between the refinement level and our metrics. Hence, we determine the correlation between refinement level (RL) and (i) mutation score (MS), (ii) the number of false positives (FP), (iii) the sensitivity to subtle faults (SF) in the 4th quartile. Due to how the refinement level is an ordinal sequence and the relationships of interest are monotonic rather than strictly linear [37], we adopt Spearman's rank correlation coefficient. For each coefficient, we additionally report a 95% confidence interval (CI), which is obtained via the Fisher z-transformation using the Bonett–Wright standard error for rank correlations. All coefficients are statistically significant ($p < 0.01$). The results are in Table XI.

TABLE XI: Spearman Correlation Coefficient between the refinement level and the proposed metrics, with 95% confidence intervals.

	Robotic Arm	Solar Panel Cleaner	RC Car
RL v MS	0.979 $[0.86, 1.00]$	0.964 $[0.67, 1.00]$	0.988 $[0.90, 1.00]$
RL v FP	-0.979 $[-1.00, -0.86]$	-0.982 $[-1.00, -0.82]$	-1.000 (n/a)
RL v SF	0.837 $[0.27, 0.97]$	0.954 $[0.60, 1.00]$	0.988 $[0.90, 1.00]$

The results show that there is a **strong positive** correlation between the number of parts modelled and the mutation score, which indicates that a higher number of parts modelled increases the number of faults detected. Conversely, there is a **strong negative** correlation between the number of false positives and the refinement of the model. This statistical correlation shows that as the model was refined the number

of false positives decreased. Lastly, the results show a strong positive correlation for all three systems (with the Robotic Arm sensitivity coefficient being the weakest at 0.837). All of the results above are statistically significant for a p-value $\leq$ 0.05.

5) Threats to validity: The mutant operators, refinement steps, and oracle thresholds (τ, ϵ) were selected by two domain experts based on prior studies and domain knowledge, but their relationship to real faults and the sensitivity of our results to these choices require further investigation. In particular, varying (τ, ϵ) could affect both mutation scores and false-positive rates, motivating future sensitivity analyses. Our false-positive analysis (RQ3) also assumes the reference implementation is correct. Although extensively tested, this assumption is not formally verified; we mitigate this threat by manually inspecting every reported failure to distinguish unreasonable inputs from genuine implementation discrepancies (Section V-B2).

Our evaluation uses search-based input generation, and it is unclear whether alternative input generation methods would produce similar results. To reduce this risk, we employ two complementary search algorithms, both achieving mutation scores of $90 - 95\%$ across the case studies.

Finally, our evaluation considers only three stationary or ground-based robotic systems of moderate complexity, and uses a cumulative refinement process that does not isolate the individual contribution of each abstraction dimension. Modelling cost is also assessed qualitatively through interviews rather than quantitative measurement. Consequently, our findings should be interpreted as domain-specific empirical evidence and practical guidance, with broader generalisation and per-dimension attribution left for future work.

VI. Guidelines and Insights for Practitioners

This section describes our recommendations for those building models for testing RAS. In this section, we first provide an overview of the guidelines and insights obtained from the interviews. Then, we provide some conclusions about the results of the experiment and devise a process around them.

Functionality abstraction is considered the least useful type of abstraction. Hence, one should model all functionality within the robot and be aware of corner cases. As for physical abstraction, its usefulness largely depends on the level of accuracy needed for the robot movement. Generally speaking, the dynamic model (dealing with forces and torque) can be abstracted if the robot does not navigate high-risk terrain or physically interact with humans. Refining the physical aspects can also be useful to compare the efficiency of different robot configurations.

The impact of temporal abstraction depends on the parts of the system that are being modelled. The advice from roboticists is to not use this type of abstraction in time-critical features. However, it can be useful when dealing with internal mechanisms such as making sure the test-model handles the ordering of messages correctly. On the other hand, abstraction of sensor data seemed to be used in almost all cases, such as not modelling raw data or employing sensor fusing. Conversely, incorporating probabilistic noise to modelled sensors is a way of making it more realistic. Lastly, regarding communications, a common type of abstraction that roboticists tend to employ is modelling the exchange of message under the assumption that no message will be lost. Contrarily, external communication between robots and other agents should be modelled with more details, as wireless networks are not as reliable.

As for the controlled experiment, the results indicate a trend between the types of abstraction and the efficiency and precision of the test suites. They show that the absence of modelled functionality leads to a low fault detection rate. Analogously, the presence of false positives indicates that the current test-model is too abstract and, hence, behaves in ways that are not expected. Therefore, reducing physical, temporal, communication, and data abstraction results in less false positives.

As a way to provide a systematic approach to act on the insights provided by the qualitative and quantitative data, we propose a process that promotes a step-wise reduction of abstraction. The process focuses on (i) reducing functionality abstraction in order to increase the fault detection capabilities of the test-model and (ii) reducing other types of abstraction to reduce the number of false positives. Figure 2 depicts the model-refinement process. The main loop of the process is to add all of the intended system functionality whilst gradually reducing some of the other forms of abstraction.

The process starts with an initial test-model that covers the most basic functionality of the SUT. This test-model serves as input to standard MBT strategies, where a test suite is generated and executed, and the verdict is examined. In the presence of false positives, we refine the model by reducing physical, temporal, communication and data abstraction. For instance, physical abstraction can be handled by factoring in kinematics and dynamics equations to the robot components. This step is repeated until false positives cannot be further reduced. Next, new functionality is modelled and, in the presence of false positives, the other abstraction types are refined. The final test-model is obtained when all desired functionality has been modelled and false positives cannot be further reduced.

VII. Conclusion

This work provides an empirical study of test-model effectiveness for RAS. We achieve this by presenting insights from interviews with roboticists and identifying the different types of abstraction and their usefulness. Then, we conduct a controlled experiment to gather empirical data on this topic. The results of our experiments indicate that the less abstract the test-model, the more effective it is. We have shown that there is a strong correlation between the refinement level and the number of faults detected and between the refinement level and the decrease in number of false positives. Lastly, we make use of the insights and empirical data to provide guidelines for practitioners who build models for testing RAS.

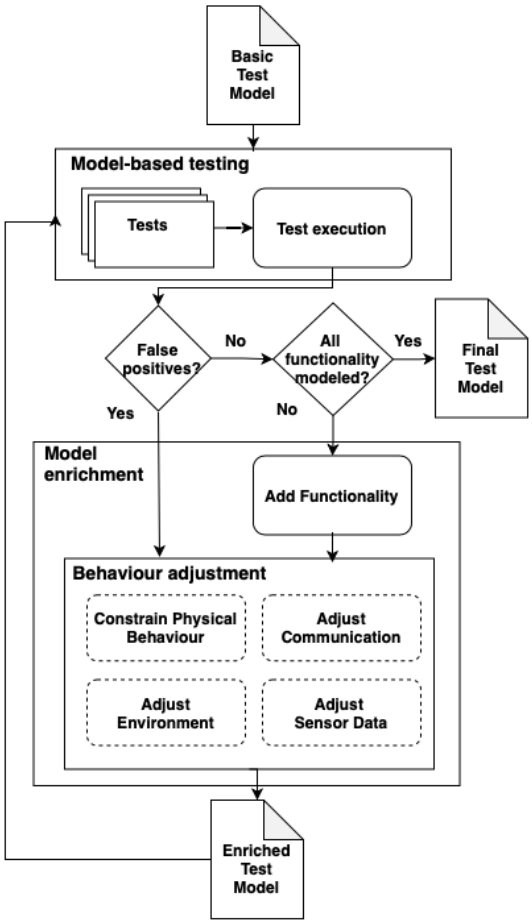

Fig. 2: Refinement process

a) Acknowledgements.: The authors have been partially supported by the UKRI TAS Node on Verifiability (EP/V026801/2). Araujo and Mousavi have been partially supported by ITEA/InnovateUK project GreenCode (600643) .

REFERENCES

[1] A. Gotlieb, D. Marijan, and H. Spieker, "Testing industrial robotic systems: A new battlefield!" *Software Engineering for Robotics*, pp. 109–137, 2021.

[2] J. Clarke, J. J. Dolado, M. Harman, R. Hierons, B. Jones, M. Lumkin, B. Mitchell, S. Mancoridis, K. Rees, M. Roper *et al.*, "Reformulating software engineering as a search problem," *IEE Proceedings-software*, vol. 150, no. 3, pp. 161–175, 2003.

[3] M. Utting, A. Pretschner, and B. Legeard, "A taxonomy of model-based testing approaches," *Software testing, verification and reliability*, vol. 22, no. 5, pp. 297–312, 2012.

[4] H. Araujo, M. R. Mousavi, and M. Varshosaz, "Testing, validation, and verification of robotic and autonomous systems: A systematic review," *ACM Trans. Softw. Eng. Methodol.*, jun 2022. [Online]. Available: https://doi.org/10.1145/3542945

[5] W. Prenninger and A. Pretschner, "Abstractions for model-based testing," *Electronic Notes in Theoretical Computer Science*, vol. 116, pp. 59–71, 2005.

[6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.220.4598.671

[7] M. Mitchell, *An introduction to genetic algorithms*. MIT press, 1998.

[8] J. Tretmans, "Model based testing with labelled transition systems," in *Formal methods and testing*. Springer, 2008, pp. 1–38.

[9] A. Cavalcanti, J. Baxter, R. Hierons, and R. Lefticaru, "Testing Robots using CSP," in *Tests and Proofs*, D. Beyer and C. Keller, Eds. Springer, 2019, pp. 21–38. [Online]. Available: papers/CBHL19.pdf

[10] A. Aerts, M. A. Reniers, and M. R. Mousavi, "Model-based testing of cyber-physical systems," in *Cyber-physical Systems*. Elsevier, 2016.

[11] T. Dreossi, T. Dang, A. Donzé, J. Kapinski, X. Jin, and J. V. Deshmukh, "Efficient guiding strategies for testing of temporal properties of hybrid systems," in *NASA Formal Methods Symposium*. Springer, 2015, pp. 127–142.

[12] B. Hoxha, H. Bach, H. Abbas, A. Dokhanchi, Y. Kobayashi, and G. Fainekos, "Towards formal specification visualization for testing and monitoring of cyber-physical systems," in *Int. Workshop on Design and Implementation of Formal Tools and Systems*, 2014.

[13] N. Khakpour and M. R. Mousavi, "Notions of conformance testing for cyber-physical systems: Overview and roadmap," in *LIPIcs-Leibniz International Proceedings in Informatics*, vol. 42. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2015.

[14] A. Afzal, D. S. Katz, C. Le Goues, and C. S. Timperley, "Simulation for robotics test automation: Developer perspectives," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2021, pp. 263–274.

[15] S. Khatiri, S. Panichella, and P. Tonella, "A roadmap for simulation-based testing of autonomous cyber-physical systems: Challenges and future direction," *ACM Transactions on Software Engineering and Methodology*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3711906

[16] A. Ortega, S. Parra, S. Schneider, and N. Hochgeschwender, "Composable and executable scenarios for simulation-based testing of mobile robots," *Frontiers in Robotics and AI*, vol. 11, 2024. [Online]. Available: https://doi.org/10.3389/frobt.2024.1363281

[17] A. Cavalcanti, W. Barnett, J. Baxter, G. Carvalho, M. C. Filho, A. Miyazawa, P. Ribeiro, and A. Sampaio, "Robostar technology: A roboticist's toolbox for combined proof, simulation, and testing," *Software Engineering for Robotics*, pp. 249–293, 2021.

[18] A. Miyazawa, P. Ribeiro, W. Li, A. Cavalcanti, J. Timmis, and J. Woodcock, "RoboChart: modelling and verification of the functional behaviour of robotic applications," *Software & Systems Modeling*, vol. 18, no. 5, pp. 3097–3149, 2019. [Online]. Available: https://rdcu.be/bh7dI

[19] A. Cavalcanti, A. Sampaio, A. Miyazawa, P. Ribeiro, M. C. Filho, A. Didier, W. Li, and J. Timmis, "Verified simulation for robotics," *Science of Computer Programming*, vol. 174, pp. 1–37, 2019. [Online]. Available: https://www-users.cs.york.ac.uk/~alcc/publications/papers/CSMRCD19.pdf

[20] E. Rohmer, S. P. N. Singh, and M. Freese, "Coppeliasim (formerly v-rep): a versatile and scalable robot simulation framework," in *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013, www.coppeliarobotics.com.

[21] A. Windisch and N. Al Moubayed, "Signal generation for search-based testing of continuous systems," in *Software Testing, Verification and Validation Workshops, 2009. ICSTW'09. International Conference on*. IEEE, 2009, pp. 121–130.

[22] M. Harman, P. McMinn, J. T. De Souza, and S. Yoo, "Search based software engineering: Techniques, taxonomy, tutorial," in *Empirical software engineering and verification*. Springer, 2010, pp. 1–59.

[23] M. Dorigo and M. Birattari, *Ant colony optimization*. Springer, 2010.

[24] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. IEEE, 1995, pp. 1942–1948.

[25] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074

[26] S. Bhavikatti, *Finite element analysis*. New Age International, 2005.

[27] J. Z. Sasiadek, "Sensor fusion," *Annual Reviews in Control*, vol. 26, no. 2, pp. 203–228, 2002.

[28] International Organization for Standardization, "ISO 8373:2021 Robotics — Vocabulary," Geneva, Switzerland, 2021, standard.

[29] G. Aravind, G. Vasan, G. Kumar, and N. B. G. Ilango, "A control strategy for an autonomous robotic vacuum cleaner for solar panels," in *Texas Instruments India Educators' Conference*, 2014, pp. 53–61.

[30] H. Araujo, G. Carvalho, M. Mousavi, and A. Sampaio, "Multi-objective search for effective testing of cyber-physical systems," in *Proceedings of the 17th International Conference on Software Enginnering and Formal Methods*. Springer, 2019.

[31] H. Abbas, B. Hoxha, G. E. Fainekos, J. V. Deshmukh, J. Kapinski, and K. Ueda, "Conformance testing as falsification for cyber-physical systems," in *Proceedings of the ACM/IEEE 5th International Conference on Cyber-Physical Systems (ICCPS 2014)*. IEEE, 2014, p. 211.

[32] H. Abbas, H. Mittelmann, and G. Fainekos, "Formal property verification in a conformance testing framework," in *2014 Twelfth ACM/IEEE Conference on Formal Methods and Models for Codesign (MEMOCODE)*. IEEE, 2014, pp. 155–164.

[33] R. Feldt, S. Poulding, D. Clark, and S. Yoo, "Test set diameter: Quantifying the diversity of sets of test cases," in *2016 IEEE international conference on software testing, verification and validation (ICST)*. IEEE, 2016, pp. 223–233.

[34] L. T. M. Hanh and N. T. Binh, "Mutation operators for simulink models," in *2012 Fourth International Conference on Knowledge and Systems Engineering*, 2012, pp. 54–59.

[35] E. Bartocci, L. Mariani, D. Ničković, and D. Yadav, "Fim: fault injection and mutation for simulink," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 1716–1720. [Online]. Available: https://doi.org/10.1145/3540250.3558932

[36] Y. Jia and M. Harman, "Higher order mutation testing," *Information and Software Technology*, vol. 51, no. 10, pp. 1379–1393, 2009.

[37] N. S. Chok, "Pearson's versus spearman's and kendall's correlation coefficients for continuous data," Ph.D. dissertation, University of Pittsburgh, 2010.